Publishing Topics

1. Experiment Objective

In this experiment, you will learn how to create a ROS2 topic publisher on the MicroROS-V2 control board using the micro-ROS framework. Through this example, you will master the micro-ROS node creation workflow, publisher initialization methods, use of timer callbacks, and the basic configuration of Wi-Fi network communication. This example publishes a self-incrementing `Int32` integer message to the topic `/publisher` at a publish rate of 1 Hz.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
micro-ROS Agent	A proxy program running on the ROS2 host side, responsible for bridging communication between the ESP32 firmware and the ROS2 system, connecting with the microcontroller via the UDP protocol
rclc	A ROS2 client library for microcontrollers, providing lightweight node, publisher, subscriber, timer and other APIs, adapted to resource-constrained embedded devices
publisher	The publisher, the message-sending end in ROS2 communication, publishes messages to a specified topic for other ROS2 nodes to subscribe to and consume
QoS (Best Effort)	The "best effort" mode in the quality-of-service policy, which does not guarantee message delivery and ordering, suitable for scenarios with high real-time requirements but tolerant of a small amount of packet loss (such as sensor data)
UDP transport	The default underlying transport protocol of micro-ROS, implementing lightweight communication over UDP, suitable for Wi-Fi network environments
rmw_uros	The RMW (ROS Middleware) implementation layer of micro-ROS, providing communication management functions such as UDP address configuration and Agent liveness detection
executor	The executor, an event-driven scheduler in micro-ROS, responsible for polling timer and subscriber handles and invoking the corresponding callback functions
timer callback	The timer callback function, automatically invoked by the executor at a configured period, used to execute tasks periodically (such as publishing messages on a timer)

2. Technical Architecture

This experiment builds a one-way data channel from an ESP32 firmware publisher to the ROS2 host:

- **Hardware layer**: The MicroROS-V2 control board (ESP32) connects to the local network via Wi-Fi
- **Transport layer**: Uses the UDP protocol, communicating with the ROS2 host through the micro-ROS Agent

- **Node configuration**: Node name `esp32_publisher`, publishing to topic `/publisher`
- **Message type**: `std_msgs/msg/Int32`, carrying a 32-bit signed integer
- **QoS mode**: Best Effort, adapting to the unreliable characteristics of the Wi-Fi UDP link
- **Data flow**: Timer (1 Hz period) → timer callback → `rcl_publish()` → micro-ROS stack → UDP → Agent → ROS2 topic bus

3. Core Topic Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
`/publisher`	`std_msgs/msg/Int32`	Best Effort	Incrementing integer published by the ESP32 timer, 1 Hz period

Usage Example:

```
# View the published incrementing integer
ros2 topic echo /publisher
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

> Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

`RMW_UXRCE_TRANSPORT`	`udp`	UDP transport (Wi-Fi communication)
`RMW_UXRCE_MAX_PUBLISHERS`	`6`	Maximum number of publishers
`RMW_UXRCE_MAX_SUBSCRIPTIONS`	`6`	Maximum number of subscribers
`RMW_UXRCE_MAX_NODES`	`1`	Maximum number of nodes
`RMW_UXRCE_MAX_SERVICES`	`0`	Services not supported
`RMW_UXRCE_MAX_CLIENTS`	`0`	Clients not supported
`RMW_UXRCE_MAX_HISTORY`	`1`	History depth = 1, no message queuing
`UCLIENT_UDP_TRANSPORT_MTU`	`4096`	Maximum UDP packet size 4 KB
`UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL`	`1`	Agent heartbeat interval 1 ms

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0, MAX_CLIENTS=0` means only topic communication is supported, not Service/Client
- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

3. Experiment Steps

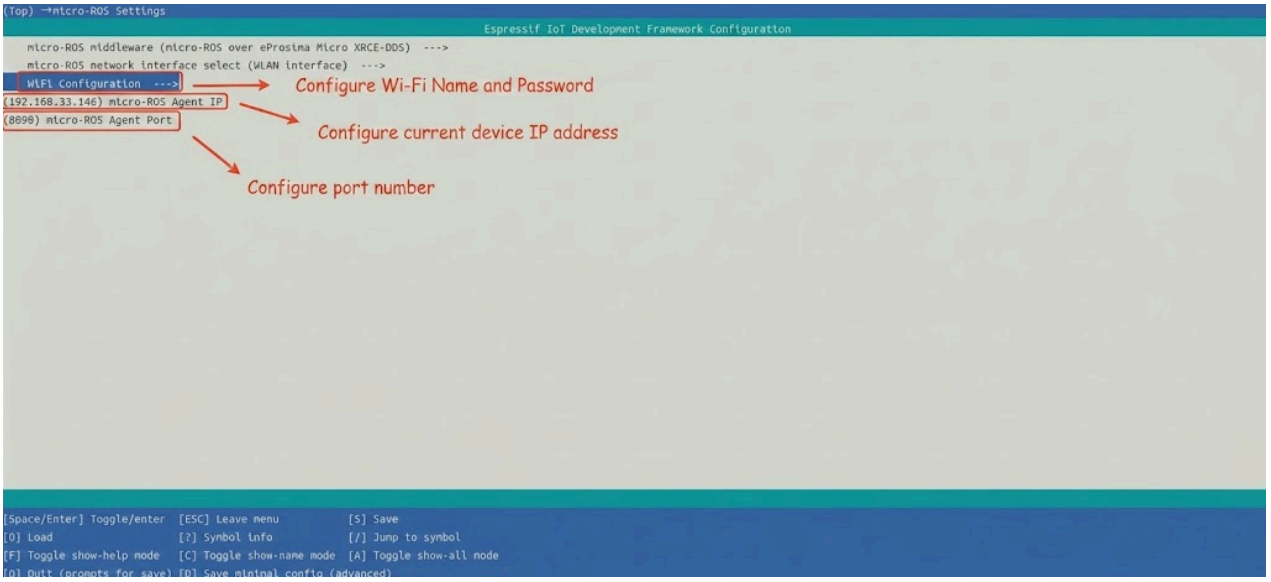
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✅ Required
USB Type-C cable	✅ Required
Wi-Fi network (2.4 GHz)	✅ Required

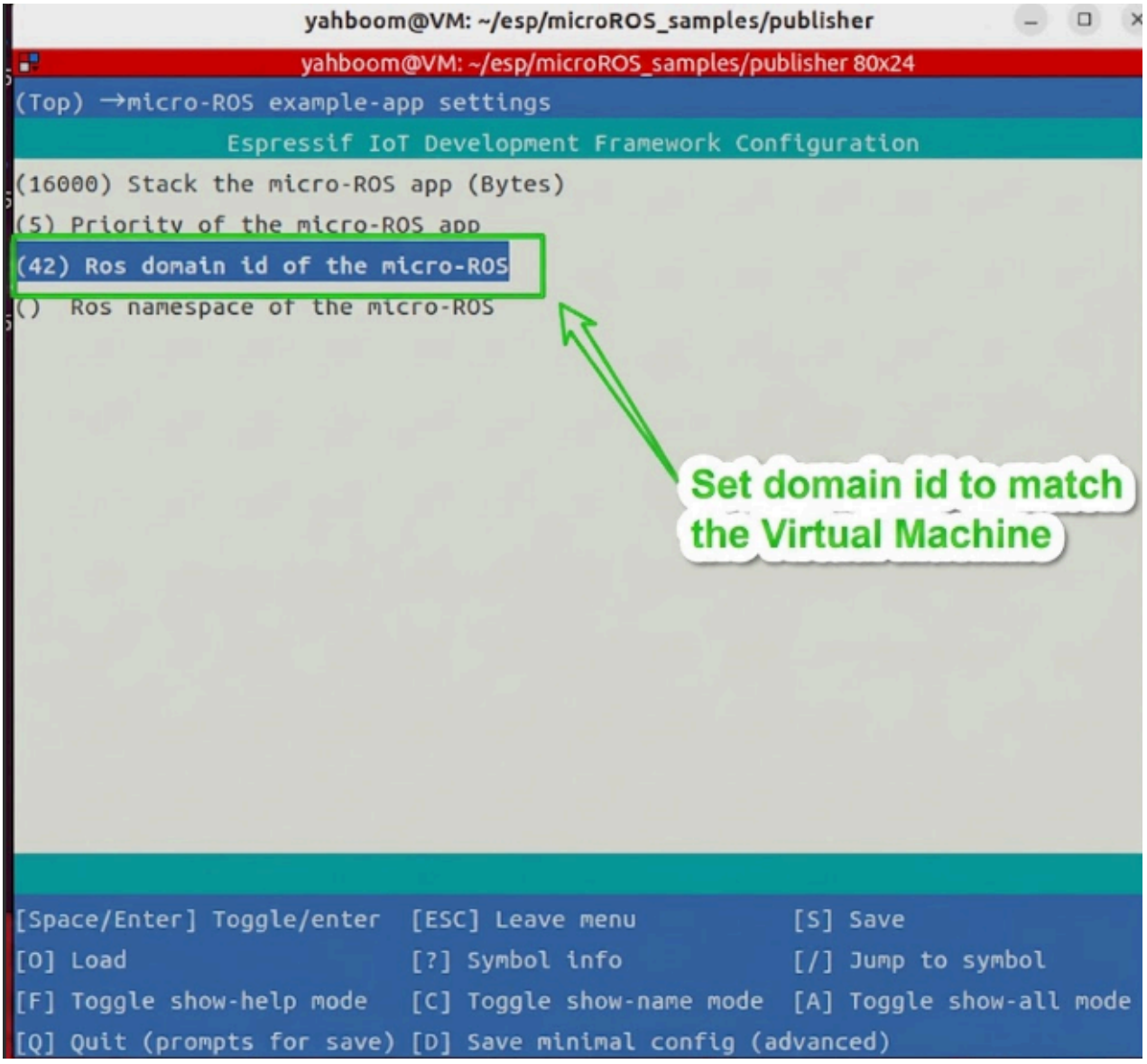
2. Prerequisites

1. The ESP32-IDF development environment has been set up
2. The MicroROS-V2 control board is connected to the computer via USB
3. Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/publisher
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



4. Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/publisher
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

1. Press `Ctrl+]` in Terminal 2 to exit the serial monitor (stop firmware execution)
2. Press `Ctrl+C` in Terminal 1 to stop micro_ros_agent

4. Experiment Result

After the build and flash succeed, the serial monitor will output:

```
I (xxx) MAIN: Agent reachable: 192.168.x.x:8888
I (xxx) MAIN: free heap before support init: xxx
I (xxx) MAIN: stack watermark before support init: xxx
I (xxx) MAIN: free heap after support init: xxx
I (xxx) MAIN: stack watermark after support init: xxx
I (xxx) MAIN: micro-ROS publisher init done
I (xxx) MAIN: Publishing: 0
I (xxx) MAIN: Publishing: 1
I (xxx) MAIN: Publishing: 2
...
```

On the ROS2 host, the topic publication can be verified with the following command:

```
ros2 topic echo /publisher
```

Expected output is an integer that increments every 1 second:

```
data: 0
---
data: 1
---
data: 2
---
```

5. Program Analysis

> Source code paths:
>
> - Firmware source code: `~/esp/microROS_samples/publisher/main/main.c`

1. Main Program Logic

`main/main.c` is the program entry point, and the flow is as follows:

1. `app_main()` calls `uros_network_interface_initialize()` to initialize the Wi-Fi network and disables Wi-Fi power saving mode to reduce UDP jitter
2. Creates the `micro_ros_task` task, in which:
    - Configures the UDP address of the micro-ROS Agent and the ROS Domain ID
    - Waits for the Agent to be reachable via `rmw_uros_ping_agent_options()`
    - Initializes the rclc support layer (`rclc_support_init_with_options`)

- Creates the ROS2 node `esp32_publisher`
- Initializes the publisher in Best Effort mode, with message type `std_msgs/msg/Int32` and topic name `publisher`
- Creates a 1 Hz timer bound to the timer callback `timer_callback`
- Creates an executor and adds the timer to it
- Calls `rclc_executor_spin_some()` in the main loop to drive the executor

2. Key Components

Publisher Configuration:

> Parameter definition file: `~/esp/microROS_samples/publisher/main/main.c`

Configuration Item	Value	Description
Node name	`esp32_publisher`	ROS2 node name
Topic name	`publisher`	Name of the published topic
Message type	`std_msgs/msg/Int32`	32-bit integer message
Publish frequency	1 Hz	Controlled by the timer `PUBLISH_PERIOD_MS=1000`
QoS mode	Best Effort	Reduces pressure on the Wi-Fi UDP link

Timer callback function `timer_callback`:

> Source file: `~/esp/microROS_samples/publisher/main/main.c`

```
static void timer_callback(rcl_timer_t *timer, int64_t last_call_time)
{
    RCLC_UNUSED(last_call_time);

    if (timer == NULL) {
        return;
    }

    rcl_ret_t rc = rcl_publish(&publisher, &msg, NULL);
    if (rc == RCL_RET_OK) {
        pub_count++;
        ESP_LOGI(TAG, "Publishing: %ld", (long)msg.data);
        msg.data++;
    } else {
        pub_fail_cnt++;
        // Print the error once every 20 failures to avoid flooding the serial port
        if ((pub_fail_cnt % 20U) == 1U) {
            print_rcl_error("rcl_publish(&publisher, &msg, NULL)", __LINE__, rc);
        }
    }
}
```

micro-ROS main task `micro_ros_task`:

> Source file: `~/esp/microROS_samples/publisher/main/main.c`

```
static void micro_ros_task(void *arg)
{
    (void)arg;

    rcl_allocator_t allocator = rcl_get_default_allocator();

    rclc_support_t support;
    memset(&support, 0, sizeof(support));

    rcl_node_t node = rcl_get_zero_initialized_node();

    rclc_executor_t executor;
    memset(&executor, 0, sizeof(executor));

    rcl_timer_t timer = rcl_get_zero_initialized_timer();
    publisher = rcl_get_zero_initialized_publisher();
    memset(&msg, 0, sizeof(msg));

    rcl_init_options_t init_options = rcl_get_zero_initialized_init_options();
    RCCHECK(rcl_init_options_init(&init_options, allocator));
    RCCHECK(rcl_init_options_set_domain_id(&init_options, ROS_DOMAIN_ID));

    rmw_init_options_t *rmw_options = rcl_init_options_get_rmw_init_options(&init_options);
    RCCHECK(rmw_uros_options_set_udp_address(ROS_AGENT_IP, ROS_AGENT_PORT, rmw_options));

    // 1. Ping the agent first; do not repeatedly re-init the support layer in a loop
    while (RMW_RET_OK != rmw_uros_ping_agent_options(
                                AGENT_PING_TIMEOUT_MS,
                                AGENT_PING_ATTEMPTS,
                                rmw_options)) {
        ESP_LOGW(TAG, "Waiting agent: %s:%s", ROS_AGENT_IP, ROS_AGENT_PORT);
        vTaskDelay(pdMS_TO_TICKS(AGENT_RETRY_DELAY_MS));
    }
    ESP_LOGI(TAG, "Agent reachable: %s:%s", ROS_AGENT_IP, ROS_AGENT_PORT);

    // 2. Initialize the rclc support layer
    RCCHECK(rclc_support_init_with_options(&support, 0, NULL, &init_options, &allocator));

    // 3. Create the ROS2 node
    RCCHECK(rclc_node_init_default(&node, "esp32_publisher", ROS_NAMESPACE, &support));

    // 4. Create the Best Effort publisher
    RCCHECK(rclc_publisher_init_best_effort(
        &publisher,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Int32),
        "publisher"));

    // 5. Create a 1 Hz timer
```

```
    RCCHECK(rclc_timer_init_default(
        &timer,
        &support,
        RCL_MS_TO_NS(PUBLISH_PERIOD_MS),
        timer_callback));

    // 6. Create an executor and add the timer to it
    RCCHECK(rclc_executor_init(&executor, &support.context, EXECUTOR_HANDLE_COUNT,
&allocator));
    RCCHECK(rclc_executor_set_timeout(&executor, RCL_MS_TO_NS(EXECUTOR_TIMEOUT_MS)));
    RCCHECK(rclc_executor_add_timer(&executor, &timer));

    msg.data = 0;
    ESP_LOGI(TAG, "micro-ROS publisher init done");

    // 7. Main loop: drive the executor
    while (1) {
        RCSOFTCHECK(rclc_executor_spin_some(&executor, RCL_MS_TO_NS(EXECUTOR_TIMEOUT_MS)));
        vTaskDelay(pdMS_TO_TICKS(EXECUTOR_LOOP_DELAY_MS));
    }
}
```

micro-ROS initialization flow:

The `micro_ros_task` above demonstrates the complete micro-ROS node initialization flow (the key steps are marked with the comments `// 1.`~`// 7.` in the code):

1. Ping the Agent — `rmw_uros_ping_agent_options` (timeout 1000 ms, at most 3 attempts, retry after a 1000 ms delay on failure)
2. Initialize the support layer — `rclc_support_init_with_options`
3. Create the node — `rclc_node_init_default("esp32_publisher")`
4. Create the publisher — `rclc_publisher_init_best_effort` (topic name `publisher`, message type `std_msgs/msg/Int32`)
5. Create the timer — `rclc_timer_init_default` (period `PUBLISH_PERIOD_MS = 1000 ms`, callback `timer_callback`)
6. Create the executor — `rclc_executor_init` + `rclc_executor_add_timer` (handle count = 1, timeout 20 ms)
7. Main loop — `rclc_executor_spin_some` drives the executor, with `EXECUTOR_LOOP_DELAY_MS = 1 ms` between each iteration

Error handling macros:

Macro	Behavior
`RCCHECK(fn)`	Checks the return value, prints an error and deletes the current task on failure
`RCSOFTCHECK(fn)`	Checks the return value, only prints an error on failure without interrupting execution

6. Common Problems

Problem	Cause	Solution
Compilation fails, header file not found	The ESP-IDF environment is not set up correctly	Run the `source ~/esp/esp-idf/export.sh` command to initialize the ESP-IDF environment
Continuously outputs "Waiting agent" after flashing	The Agent is not started or the IP/port configuration is wrong	Check whether the Agent on the ROS2 host is started, and confirm that the Wi-Fi SSID/password and Agent IP are correctly configured in menuconfig
`ros2 topic echo` receives no messages	QoS mismatch or inconsistent topic name	Confirm the topic name is `/publisher` and that the Agent and the control board are on the same Wi-Fi network
Serial port outputs a large number of publish failure logs	Poor Wi-Fi signal or unstable Agent connection	The failure logs are rate-limited (printed once every 20 failures); check the Wi-Fi signal strength